

Architecture et Programmation Assembleur - Contrôle Continu

 Classe: B2A Durée: 2h Semestre 2 (2025-2026) Campus Keyce

Exercice 1 : Fondamentaux de la Machine

4
points

1.1. Conversions (2 points)

a) 11100101_2 en hexadécimal $E5_{16}$

$1110_2 = E_{16}$ et $0101_2 = 5_{16}$. Donc
 $11100101_2 = E5_{16}$.

b) $A7_{16}$ en binaire $1010\ 0111_2$

$A_{16} = 1010_2$ et $7_{16} = 0111_2$. Donc $A7_{16}$
 $= 10100111_2$.

c) 200_{10} en binaire sur 8 bits $1100\ 1000_2$

Commentaire : 200 n'est pas
représentable en non-signé sur 8
bits. Sur 8 bits, cela correspondrait
à un nombre négatif en
complément à 2 car le bit de poids

d) -15_{10} en complément à 2 (8 bits) $1111\ 0001_2$

$+15 = 0000\ 1111_2$
Complément à 1 = $1111\ 0000_2$
 $+ 1 = 1111\ 0001_2$

fort est à 1 (débordement si l'on considère signé).

1.2. Opérations logiques (2 points)

Données : $AL = 10110101_2$ (B5h)

1) `AND AL, 0Fh`

→ $0000\ 0101_2$ (05h)

Masquage des 4 bits de poids fort (mis à 0), les 4 bits de poids faible sont conservés.

2) `OR AL, 0F0h`

→ $1111\ 0101_2$ (F5h)

Les 4 bits de poids fort sont forcés à 1, les 4 de poids faible restent inchangés.

3) `XOR AL, 0FFh`

→ $0100\ 1010_2$ (4Ah)

Inverse tous les bits de AL.
Équivalent à un complément à 1 (NOT AL).

4) `SHL AL, 1`

→ $0110\ 1010_2$ (6Ah)

Décalage à gauche d'un bit. Le bit sortant (1) va dans le Carry Flag (CF). Un 0 entre à droite.



Exercice 2 : Architecture du Processeur

4
points

2.1. Registre d'état EFLAGS (2 points)

Après l'instruction `CMP EAX, EBX` (qui effectue $EAX - EBX$ sans modifier EAX).

a) $EAX = 10$, $EBX = 10$

b) $EAX = 5$, $EBX = 10$

Zéro Flag (ZF) = 1 car $10 - 10 = 0$.

Indique l'égalité.

Sign Flag (SF) = 0 car le résultat n'est pas négatif.

Carry Flag (CF) = 0 pas d'emprunt.

Signification : Les deux opérandes sont égaux.

Sign Flag (SF) = 1 car $5 - 10 = -5$ (négatif).

Carry Flag (CF) = 1 car la soustraction d'un nombre plus grand nécessite un emprunt (borrow).

Zéro Flag (ZF) = 0 car ce n'est pas 0.

Signification : EAX est strictement inférieur à EBX.

2.2. Registres spécialisés (2 points)

Instruction

Modification principale

JMP label

Modifie **EIP** (Instruction Pointer) pour pointer vers l'adresse du label.

**CALL
fonction**

Modifie **EIP** (saut vers la fonction) et **ESP** (Stack Pointer) pour empiler l'adresse de retour.

PUSH EAX

Modifie **ESP** (décrémenté de 4 octets) et modifie la mémoire pointée par ESP.

**ADD EAX,
EBX**

Modifie **EAX** (qui reçoit la somme) et les **EFLAGS** (ZF, SF, CF, OF, etc.).



Exercice 3 : Jeu d'Instructions

5 points

3.1. Instructions arithmétiques (2 points)

#	Instruction	Action & Contenu final des registres
1	<code>MOV EAX, 50</code>	Copie la valeur 50 dans EAX. EAX = 50.
2	<code>MOV EBX, 7</code>	Copie la valeur 7 dans EBX. EBX = 7.
3	<code>XOR EDX, EDX</code>	Met EDX à 0 (l'extension pour la division signée). EDX = 0.
4	<code>DIV EBX</code>	Divise EDX:EAX par EBX (50 / 7). Le quotient va dans EAX, le reste dans EDX. EAX = 7 (Quotient) EDX = 1 (Reste de $50 = 7 * 7 + 1$)

3.2. Structure conditionnelle (3 points)

Traduction de : `if (x > y) { max = x; } else { max = y; }`

```

; Utilisation de CMP et sauts conditionnels
MOV EAX, [x]          ; Charger x dans EAX
CMP EAX, [y]          ; Comparer x et y
JG x_est_max          ; Si x > y (Jump if Greater), sauter à x_est_max

; Bloc else : y est plus grand ou égal
MOV EAX, [y]          ; Charger y dans EAX
JMP fin               ; Sauter l'autre bloc

x_est_max:
; Bloc if : EAX contient déjà x, rien de plus à faire
; (ou MOV EAX, [x] pour être explicite)

fin:
MOV [max], EAX        ; Stocker le plus grand dans max

```



Exercice 4 : Programmation sous SASM

7
points

Écriture d'un programme complet (Recherche d'un élément dans un tableau).

```
%include "io.inc"

section .data
    ; 1. Tableau d'entiers et variable recherche (1,5 pts)
    tableau dd 10, 15, 20, 25, 42, 25, 60, 25, 90, 100
    taille dd 10
    recherche dd 25

    msg_present db "Présent", 0
    msg_absent db "Absent", 0

section .bss
    ; 3. Variable pour stocker le nombre d'occurrences
    occurrences resd 1

section .text
global CMAIN
CMAIN:
    mov ebp, esp        ; pour le debug correct dans SASM

    ; 2. Parcours du tableau (2 pts)
    mov ecx, [taille]    ; Compteur de boucle = 10
    xor eax, eax         ; eax servira de compteur d'occurrences (0)
    mov esi, tableau     ; esi pointe sur le tableau
    mov ebx, [recherche] ; ebx contient la valeur à chercher (25)

boucle:
    cmp [esi], ebx       ; Compare l'élément courant avec 'recherche'
    jne pas_egal         ; Si non égal, on saute l'incrémentation
    inc eax              ; Si égal, on incrémente le compteur

pas_egal:
    add esi, 4           ; Avancer au prochain élément (32 bits = 4 octets)
    loop boucle          ; Décrémente ecx, boucle si != 0
```

```
; 3. Stockage et affichage des occurrences (2 pts)
mov [occurrences], eax
PRINT_DEC 4, [occurrences]
NEWLINE

; 4. Affichage conditionnel "Présent" / "Absent" (1,5 pts)
cmp eax, 0          ; Vérifie si le nombre d'occurrences est 0
je absent           ; Si ZF=1 (zéro occurrence), on saute

; Si c'est au moins 1 occurrence :
PRINT_STRING msg_present
NEWLINE
jmp fin

absent:
PRINT_STRING msg_absent
NEWLINE

fin:
xor eax, eax        ; Code de retour 0
ret
```